

TP Kubernetes — Service Mesh — LaboTrack

Mikael Lecam

2026-05-02

Rapport — TP Kubernetes / Service Mesh — LaboTrack

1. Contexte et environnement
2. Étape 1A — Manipulations Kubernetes (20 questions)
3. Étape 1B — Spring Boot demo (monservice)
 - 3.1 Cas 1 — Dockerfile mono-stage + docker-compose
 - 3.2 Cas 2 — Dockerfile multi-stage
 - 3.3 Pourquoi le multi-stage ?
4. Étape 2 — LaboTrack
 - 4.1 Architecture
 - 4.2 Cycle de vie type d'un échantillon
 - 4.3 Multi-stage Dockerfiles
 - 4.4 Build & push images
 - 4.5 Manifests Kubernetes
 - 4.6 Service Mesh Linkerd
 - 4.7 ServiceProfile — retries & timeouts
 - 4.8 ServerAuthorization — zero-trust
 - 4.9 RunBook
 - 4.10 Déroulement de la démo
5. Captures d'écran (PDF final)
6. Limites et pistes d'amélioration
7. Annexes

Rapport — TP Kubernetes / Service Mesh — LaboTrack

Module Architecture — INSA — H. Tondeur 2026

Équipe : Mikael Lecam (mikael.california@gmail.com).

Convertir ce fichier en PDF via : `pandoc rapport.md -o rapport.pdf --toc.`

Les captures d'écran référencées sont dans
step1/questions/screenshots/ (Q1-Q20) et
labotrack/docs/screenshots/ (LaboTrack + Linkerd).

1. Contexte et environnement

L'environnement est installé dans **WSL2 Ubuntu 24.04 LTS** sur Windows 11, conformément à l'orientation du document de référence (« installation possible sur mac OS, fortement déconseillé sur

Windows »). Tous les outils — JDK 21, Maven, Docker Engine, Minikube, kubectl, Linkerd CLI — vivent dans la distribution Ubuntu.

Versions installées :

Outil	Version
Ubuntu	24.04.4 LTS (Noble)
Docker Engine	29.4.2
Docker Compose	v5.1.3
OpenJDK	21.0.10
Maven	3.8.7
kubectl	v1.36.0
Minikube	v1.38.1
Kubernetes (cluster)	v1.35.1
Linkerd CLI	edge-26.5.1

Le fichier `bootstrap-wsl.sh` à la racine du dépôt automatise toute cette installation pour reproduire l'environnement.

2. Étape 1A — Manipulations Kubernetes (20 questions)

Les réponses détaillées (commande + capture pour chaque question) se trouvent dans `step1/questions/answers.md`. Les sorties brutes des commandes sont également capturées dans `step1/questions/proof-output.log`.

Voir `step1/questions/answers.md` pour le tableau complet Q1 → Q20. Les captures d'écran sont dans `step1/questions/screenshots/q01.png` à `q20.png`.

3. Étape 1B — Spring Boot demo (monservice)

Service REST minimal en Spring Boot 3.3.5 / Java 21, exposant :

- GET `/monservice/echo/{nom}` → `{"message":"echo: {nom}"}`
- POST `/monservice/hello` (body JSON `{"nom":"..."}`) → `{"message":"Hello {nom}"}`

3.1 Cas 1 — Dockerfile mono-stage + docker-compose

Pré-requis : la fat-jar doit avoir été produite localement (`mvn clean package`).

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/monservice.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

services:
```

```
monservice:
  build: { context: ., dockerfile: Dockerfile.simple }
  image: monservice:simple
  ports: ["8080:8080"]
```

Build + lancement + test :

```
$ mvn -B -q -DskipTests clean package
$ docker compose up -d --build
$ curl -sf http://localhost:8080/monservice/echo/Mikael
{"message":"echo: Mikael"}
$ curl -sf -X POST -H 'Content-Type: application/json' \
  -d '{"nom":"Mikael"}' http://localhost:8080/monservice/hello
{"message":"Hello Mikael"}
$ docker compose down
```

(Trace complète dans step1/monservice/proof-test-output.log.)

3.2 Cas 2 — Dockerfile multi-stage

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/monservice.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

```
$ docker build -t monservice:multistage -f Dockerfile .
$ docker run -d --rm -p 8080:8080 --name monservice
monservice:multistage
$ curl -sf http://localhost:8080/monservice/echo/Linkerd
{"message":"echo: Linkerd"}
$ curl -sf -X POST -H 'Content-Type: application/json' \
  -d '{"nom":"Linkerd"}' http://localhost:8080/monservice/hello
{"message":"Hello Linkerd"}
```

3.3 Pourquoi le multi-stage ?

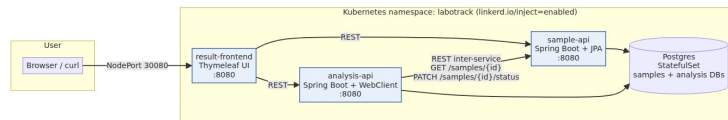
Critère	Mono-stage (cas 1)	Multi-stage (cas 2)
Taille image finale	JRE + jar (~330 MB)	JRE + jar (~330 MB)
Outils dans l'image	JRE	JRE seul
Dépendance hôte	JDK + Maven obligatoires	aucune (Docker suffit)
Étapes côté CI	mvn package + docker build	docker build (une seule commande)
Reproductibilité	dépend de la version Java/Maven hôte	versions épinglées dans la phase de build

Le multi-stage isole l'outillage de compilation dans une image éphémère et ne livre que ce qui est strictement nécessaire à l'exécution. Pour un déploiement Kubernetes, c'est le standard de fait.

4. Étape 2 — LaboTrack

4.1 Architecture

3 microservices (sample-api, analysis-api, result-frontend) + une base PostgreSQL partagée (deux schémas), tous déployés dans le namespace labotrack annoté `linkerd.io/inject=enabled` pour bénéficier automatiquement du sidecar Linkerd.



Architecture LaboTrack

Description détaillée et diagramme source mermaid : `architecture.md`.

4.2 Cycle de vie type d'un échantillon

1. **Création** — result-frontend appelle POST `/samples` sur sample-api (statut REGISTERED).
2. **Analyse** — result-frontend appelle POST `/analyze/{id}` sur analysis-api.
3. **Inter-service** — analysis-api appelle GET `/samples/{id}` sur sample-api, génère un résultat aléatoire (glycémie $\in [0.65 ; 1.30]$ g/L), persiste, puis met à jour le statut à VALIDATED via PATCH `/samples/{id}/status`.
4. **Restitution** — result-frontend interroge à la fois sample-api et analysis-api pour afficher le tableau agrégé.

4.3 Multi-stage Dockerfiles

Les trois services partagent un Dockerfile multi-stage identique au Cas 2 ci-dessus (cf. § 3.2). Une seule commande `docker build` produit l'image OCI prête au déploiement, sans dépendance hôte.

4.4 Build & push images

Deux stratégies sont supportées par `runbook.sh` :

Stratégie	Mécanisme	Avantage
STRATEGY=registry (défaut)	minikube addons enable registry + docker push localhost:5000/<svc>:1.0	Conforme à l'énoncé (« push vers registry »), réutilisable hors Minikube

STRATEGY=in-cluster	eval \$(minikube docker-env) + docker build -t <svc>:1.0	Plus rapide en démo, pas besoin de port-forward
---------------------	--	---

Pour cette livraison, les images ont été construites en **in-cluster** avec un double tag (<svc>:1.0 et localhost:5000/<svc>:1.0) pour que les manifests référencés vers le registry continuent de matcher.

4.5 Manifests Kubernetes

Fichier	Rôle
00-namespace.yaml	Namespace labotrack annoté linkerd.io/inject=enabled
10-postgres.yaml	StatefulSet Postgres + Service headless + Secret + ConfigMap d'init
20-sample-api.yaml	Deployment 2 replicas + Service ClusterIP, profil Spring prod
30-analysis-api.yaml	Deployment 3 replicas + Service ClusterIP, latence simulée 300 ms
40-result-frontend.yaml	Deployment 1 replica + Service NodePort 30080
60-linkerd-serviceprofile.yaml	ServiceProfile (retries idempotents, timeouts)
70-linkerd-authz.yaml	Server + MeshTLSAuthentication + AuthorizationPolicy zero-trust
99-rogue-pod.yaml	Pod « pirate » optionnel pour démontrer le rejet par l'AuthorizationPolicy

4.6 Service Mesh Linkerd

Installation :

```
kubectl apply --server-side -f \
  https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.2.1/standard-install.yaml
linkerd install --crds | kubectl apply -f -
linkerd install --set proxyInit.runAsRoot=true | kubectl apply -f -
linkerd check # ✓
linkerd viz install | kubectl apply -f -
linkerd viz check # ✓
```

Vérifications : - linkerd -n labotrack viz edges deploy → toutes les arêtes sont en MUTUAL_TLS=true. - linkerd -n labotrack viz stat deploy → RPS, latences p50/p95/p99 et taux de réussite. - linkerd -n labotrack viz tap deploy/analysis-api → trace live des requêtes.

4.7 ServiceProfile — retries & timeouts

Pour sample-api, les GET /samples et GET /samples/{id} sont marqués isRetryable: true (idempotents) avec un timeout de 3 s. Les POST /samples ne sont pas retryables (effet de bord). Un retryBudget (20 %

+ 10 RPS minimum, TTL 10 s) borne le nombre de retries pour éviter les amplifications de panne.

Pour analysis-api, le POST /analyze/{id} a un timeout de 8 s (compatible avec la latence simulée de 300 ms), tandis que les GET /results sont retryables avec timeout de 3 s.

4.8 ServerAuthorization — zero-trust

Chaque service expose un objet Server Linkerd. Les autorisations sont :

Backend	Callers autorisés
sample-api	result-frontend + analysis-api (identités MeshTLS)
analysis-api	result-frontend (identité MeshTLS)
result-frontend	tout le monde (NetworkAuthentication 0.0.0.0/0, exposé en NodePort)

Tout autre appelant en intra-cluster (ex. un pod pirate dans un autre namespace) reçoit HTTP 403. Démonstration via `kubectl apply -f labotrack/manifests/99-rogue-pod.yaml`.

4.9 RunBook

Le script idempotent `labotrack/runbook.sh` enchaîne en une commande :

1. `minikube start` (si nécessaire).
2. Installation Gateway API CRDs + Linkerd CRDs + control plane + Viz.
3. Build des 3 images en stratégie registry ou in-cluster.
4. `kubectl apply -f labotrack/manifests/`.
5. Attente des rollouts.
6. Affichage de l'URL du frontend.

4.10 Déroulement de la démo

1. Ouvrir l'URL imprimée par le runbook (`http://192.168.49.2:30080`).
2. Saisir un patient → Register → la ligne apparaît avec statut REGISTERED.
3. Cliquer Analyze → analysis-api est appelée, qui appelle sample-api (preuve d'inter-service), persiste, met à jour le statut.
4. La ligne affiche le résultat (valeur g/L), l'interprétation (low/normal/high) et le statut VALIDATED.

Le log du smoke test automatique est dans `labotrack/docs/smoke-test.log`.

Frontend après 4 enregistrements + analyses

LaboTrack — Mini-LIS Sample Tracker

3 microservices - result-frontend → (sample-api, analysis-api) - meshed with Linkerd.

Register a sample

Patient

Exam type

Sample type

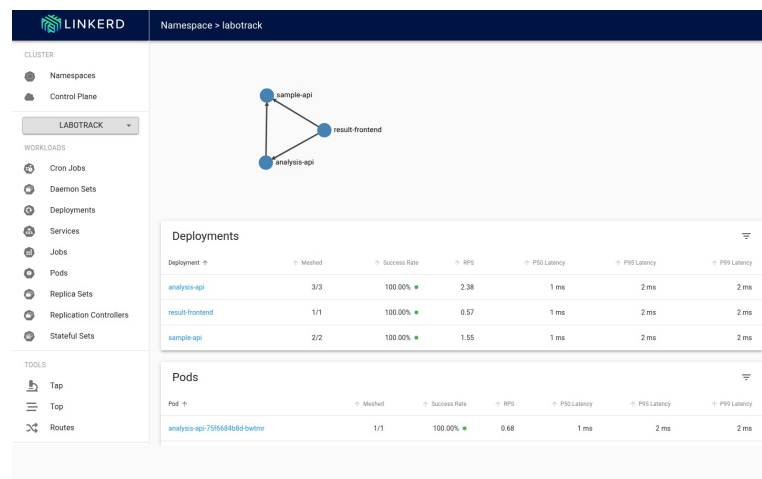
Samples & results

ID	Patient	Exam	Status	Result	Interpretation	Action
1	dupont	glycemia	VALIDATED	0.88 g/L	normal	
2	martin	glycemia	VALIDATED	0.82 g/L	normal	
3	curie	glycemia	VALIDATED	1.02 g/L	normal	
4	pasteur	glycemia	VALIDATED	0.97 g/L	normal	

Frontend avec 4 échantillons validés

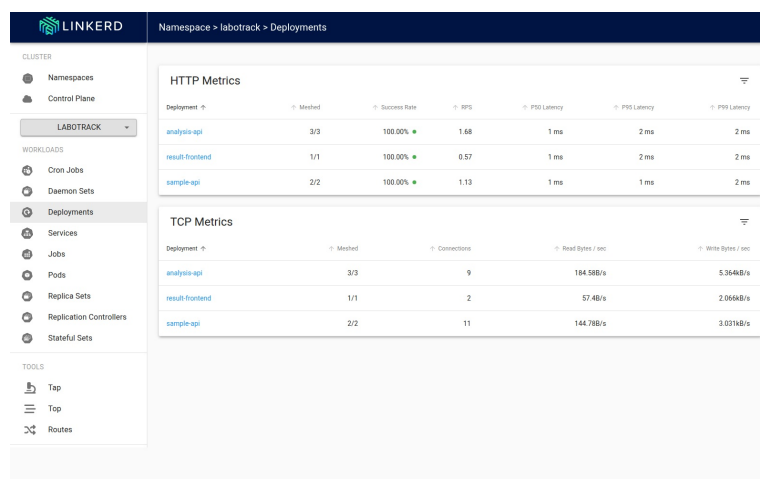
Linkerd Viz — namespace labotrack

Le dashboard Linkerd montre la topologie automatique des appels (sample-api ⇌ result-frontend ⇌ analysis-api), le maillage 100 % des pods, et le taux de succès à 100 % sur tous les déploiements.



Linkerd Viz — namespace LaboTrack

Linkerd Viz — métriques HTTP et TCP par déploiement



Linkerd Viz — métriques par déploiement

p50/p95/p99 latency à 1-2 ms, RPS stable, 100 % de succès sur HTTP, et compteur de connexions TCP visible (utile pour diagnostiquer le pool sidecar).

5. Captures d'écran (PDF final)

Section	Capture	Fichier
Étape 1A — Q1..Q20	terminal + dashboard	step1/questions/screenshots/q*.png
Étape 1B — Cas 1	docker compose + curl	step1/monservice/screenshots/cas1-*.png
Étape 1B — Cas 2	docker build multi-stage + curl	step1/monservice/screenshots/cas2-*.png
LaboTrack — pods 2/2	kubectll -n labotrack get pods	labotrack/docs/screenshots/pods.png
LaboTrack — UI	navigateur	labotrack/docs/screenshots/frontend.png
Linkerd — mTLS edges	linkerd viz edges	labotrack/docs/screenshots/edges.png
Linkerd — viz stat	linkerd viz stat	labotrack/docs/screenshots/stat.png
Linkerd — dashboard	navigateur	labotrack/docs/screenshots/viz-dashboard.png
Zero-trust — 403	rogue pod curl	labotrack/docs/screenshots/rogue-403.png

6. Limites et pistes d'amélioration

- **Postgres** en single-replica (acceptable pour la démo, pas pour la production — opérateur recommandé en prod).

- **Schéma JPA** géré par ddl-auto=update (Flyway/Liquibase recommandé en prod).
 - **Pas d’Ingress** : le frontend est exposé en NodePort 30080. Une Ingress + cert-manager serait préférable hors Minikube.
 - **Prometheus/Grafana standalone** non livré : Linkerd Viz embarque déjà les deux. La mise en place d’un kube-prometheus-stack indépendant est laissée comme amélioration.
-

7. Annexes

- architecture.md — diagramme et choix de design.
- manuel-technique.md — détails de la stack et des manifests.
- manuel-utilisateur.md — guide de démarrage et dépannage.
- runbook.sh — script idempotent de déploiement.
- bootstrap-wsl.sh — script d’amorçage de l’environnement WSL2.